

EXAMEN PRATIQUE

Arduino Uno avec Proteus ISIS

Compte rendu des quatre manipulations

Étudiant(e) : [Nom et prénom]

Année universitaire : [À renseigner]

Encadrant : M. Aymane BOUMEZZOUGH

Date de réalisation : [À renseigner]

Présentation du travail

Ce TP consiste à développer quatre petites applications autour d'une carte Arduino Uno. Elles sont d'abord programmées dans Arduino IDE, puis prévues pour être testées dans Proteus ISIS après chargement du fichier HEX. Chaque manipulation cible une notion différente : la mesure analogique, les temporisations, l'entrée numérique et la commande d'un servomoteur.

La démarche suivie est la même pour chaque partie. Le montage est câblé conformément au sujet, les broches sont déclarées dans le programme, puis le comportement attendu est comparé aux observations de la simulation. Cette comparaison permet de repérer une erreur de câblage ou de logique.

Objectifs pédagogiques

- acquérir une valeur avec `analogRead()` ;
- utiliser `digitalRead()` et `digitalWrite()` ;
- produire une séquence temporelle avec `delay()` ;
- stabiliser une entrée grâce à une résistance pull-down ;
- commander un servo avec `Servo.h` et `map()`.

Outils utilisés

Outil	Rôle dans le TP
Arduino IDE	Écriture, vérification et export du programme au format HEX
Proteus ISIS	Réalisation du schéma et observation du fonctionnement simulé
Arduino Uno R3	Carte cible commune aux quatre exercices

État du document : les programmes ont été compilés pour Arduino Uno. Les simulations Proteus n'ont pas été exécutées dans l'environnement de préparation. Les zones réservées doivent donc recevoir uniquement les captures et mesures réellement obtenues par l'étudiant.

Manipulation 1 — Détection d'un seuil

But : utiliser le potentiomètre comme entrée analogique et commander une LED rouge lorsque la mesure dépasse strictement 512.

Matériel et câblage

Composants Proteus : ARDUINO UNO R3, POT-HG, LED-RED et MINIRES 220 Ω .

Point du circuit	Raccordement
Deux extrémités du POT-HG	Une au 5 V et l'autre au GND
Curseur du POT-HG	Entrée analogique A0
Commande de la LED	D8 → résistance 220 Ω → anode de LED
Retour de la LED	Cathode vers GND

Emplacement du schéma

[Insérer la capture Proteus de l'exercice 1]
Les connexions A0, D8, 5 V et GND doivent être lisibles.

Programme employé

```
const int potentiometerPin = A0;
const int ledPin = 8;

void setup() {
  // La LED est commandée par la broche 8.
  pinMode(ledPin, OUTPUT);
}

void loop() {
  // Lecture du potentiomètre entre 0 et 1023.
  int analogValue = analogRead(potentiometerPin);

  // La LED s'allume seulement au-dessus du seuil 512.
  if (analogValue > 512) {
    digitalWrite(ledPin, HIGH);
  } else {
    digitalWrite(ledPin, LOW);
  }
}
```

Lecture du fonctionnement

Chaîne logique : position du potentiomètre → conversion analogique sur 10 bits → comparaison à 512 → choix de l'état de D8.

L'entrée A0 fournit un nombre de 0 à 1023. La condition utilise le signe « supérieur à » et non « supérieur ou égal ». Par conséquent, la mesure 512 laisse la LED éteinte ; la première valeur qui l'allume est 513.

Prévision et relevé expérimental

Valeur A0	État prévu	État observé
0 à 512	LED éteinte	[À compléter]
513 à 1023	LED allumée	[À compléter]

[Insérer une capture du résultat de simulation]

Analyse personnelle : indiquer les valeurs testées, l'instant du changement d'état et l'accord ou l'écart avec la prévision. [...]

Bilan de la manipulation 1. La solution logicielle réalise une commande tout-ou-rien à partir d'une mesure analogique. Elle respecte le seuil strict fixé par l'énoncé. La validation du basculement entre 512 et 513 est à compléter avec les observations Proteus. Difficulté réelle : [à renseigner].

Manipulation 2 — Séquence d'un feu tricolore

But : répéter automatiquement l'ordre vert, jaune, rouge avec des durées respectives de 3 s, 1 s et 3 s.

Matériel et câblage

Composants Proteus : ARDUINO UNO R3, LED-GREEN, LED-YELLOW, LED-RED et trois MINIRES 220 Ω .

Couleur	Broche	Branchement
Rouge	D13	D13 $\rightarrow$ 220 Ω $\rightarrow$ anode ; cathode $\rightarrow$ GND
Jaune	D12	D12 $\rightarrow$ 220 Ω $\rightarrow$ anode ; cathode $\rightarrow$ GND
Vert	D11	D11 $\rightarrow$ 220 Ω $\rightarrow$ anode ; cathode $\rightarrow$ GND

Emplacement du schéma

[Insérer la capture Proteus de l'exercice 2]
Identifier clairement les trois sorties et les trois résistances.

Programme employé

```
const int redLedPin = 13;
const int yellowLedPin = 12;
const int greenLedPin = 11;

void setup() {
  pinMode(redLedPin, OUTPUT);
  pinMode(yellowLedPin, OUTPUT);
  pinMode(greenLedPin, OUTPUT);
}

void loop() {
  // Feu vert pendant 3 secondes.
  digitalWrite(greenLedPin, HIGH);
  digitalWrite(yellowLedPin, LOW);
  digitalWrite(redLedPin, LOW);
  delay(3000);

  // Feu jaune pendant 1 seconde.
  digitalWrite(greenLedPin, LOW);
  digitalWrite(yellowLedPin, HIGH);
  digitalWrite(redLedPin, LOW);
  delay(1000);

  // Feu rouge pendant 3 secondes.
  digitalWrite(greenLedPin, LOW);
  digitalWrite(yellowLedPin, LOW);
  digitalWrite(redLedPin, HIGH);
  delay(3000);
}
```

Lecture du fonctionnement

Séquence : VERT/HIGH pendant 3000 ms → JAUNE/HIGH pendant 1000 ms → ROUGE/HIGH pendant 3000 ms → retour au début de `loop()`.

Chaque étape allume une couleur et force les deux autres à LOW. Cette écriture explicite empêche le maintien accidentel de deux feux. La fonction `delay()` conserve l'état pendant la durée demandée. La période du cycle complet vaut 7 secondes.

Prévision et relevé expérimental

Temps dans le cycle	Couleur prévue	Observation
0–3 s	Vert uniquement	[À compléter]
3–4 s	Jaune uniquement	[À compléter]
4–7 s	Rouge uniquement	[À compléter]

[Insérer les captures des trois phases du feu]

Analyse personnelle : relever l'ordre, les durées et la présence éventuelle d'un chevauchement entre deux couleurs. [...]

Bilan de la manipulation 2. Le programme construit une séquence cyclique à trois états et traduit les secondes en millisecondes dans `delay()`. Le contrôle final doit montrer le cycle vert-jaune-rouge sans allumage simultané. Difficulté réelle : [à renseigner].

Manipulation 3 — Éclairage commandé par bouton

But : obtenir une action non maintenue : la LED bleue est active pendant l'appui et s'éteint dès que le bouton est relâché.

Matériel et câblage

Composants Proteus : ARDUINO UNO R3, BUTTON, LED-BLUE, une MINIRES 10 kΩ et une MINIRES 220 Ω.

Élément	Raccordement
Bouton	Entre le 5 V et l'entrée D2
Pull-down	10 kΩ entre D2 et GND
LED bleue	D7 → 220 Ω → anode ; cathode → GND

Emplacement du schéma

[Insérer la capture Proteus de l'exercice 3]
Faire apparaître la résistance 10 kΩ entre D2 et la masse.

Programme employé

```
const int buttonPin = 2;
const int ledPin = 7;

void setup() {
  // Le bouton utilise une résistance pull-down externe de 10 kΩ.
  pinMode(buttonPin, INPUT);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  // La lecture vaut HIGH uniquement pendant l'appui.
  int buttonState = digitalRead(buttonPin);

  if (buttonState == HIGH) {
    digitalWrite(ledPin, HIGH);
  } else {
    digitalWrite(ledPin, LOW);
  }
}
```

Lecture du fonctionnement

Au repos : la résistance impose 0 V sur D2, donc LOW et LED éteinte.

Pendant l'appui : le bouton applique 5 V sur D2, donc HIGH et LED allumée.

La broche 2 est déclarée en INPUT, car le montage contient déjà une résistance de rappel externe. Le programme recopie le sens logique du bouton sur la LED sans mémoriser l'état précédent.

Prévision et relevé expérimental

Situation	D2 attendu	D7 attendu	Observation
Bouton libre	LOW	LOW	[À compléter]
Bouton maintenu	HIGH	HIGH	[À compléter]
Bouton relâché	LOW	LOW	[À compléter]

[Insérer les captures bouton relâché et bouton appuyé]

Analyse personnelle : décrire la réaction pendant l'appui et au relâchement. Indiquer si l'entrée reste stable au repos. [...]

Bilan de la manipulation 3. Le montage illustre le rôle essentiel du pull-down : fournir un niveau bas défini lorsque le contact est ouvert. Le résultat recherché n'est pas un interrupteur mémorisé, mais un éclairage limité à la durée de l'appui. Difficulté réelle : [à renseigner].

Manipulation 4 — Positionnement d'un servomoteur

But : faire correspondre la course 0–1023 du potentiomètre à une consigne angulaire allant de 0° à 180°.

Matériel et câblage

Composants Proteus : ARDUINO UNO R3, POT-HG et MOTOR-SERVO.

Élément	Raccordement
Potentiomètre	Extrémités au 5 V et GND ; curseur sur A0
Signal du servo	Broche PWM D9
Alimentation du servo	VCC au 5 V et masse au GND commun

En montage réel, une alimentation 5 V séparée peut être nécessaire pour le courant du servomoteur. Sa masse doit être commune avec celle de l'Arduino.

Emplacement du schéma

[Insérer la capture Proteus de l'exercice 4]
Montrer A0, D9, les alimentations et la masse commune.

Programme employé

```
#include <Servo.h>

const int potentiometerPin = A0;
const int servoPin = 9;

Servo myServo;

void setup() {
  // Association du signal du servomoteur à la broche 9.
  myServo.attach(servoPin);
}

void loop() {
  // Lecture de la position du potentiomètre.
  int analogValue = analogRead(potentiometerPin);

  // Conversion de 0-1023 vers un angle de 0-180 degrés.
  int angle = map(analogValue, 0, 1023, 0, 180);

  // Application de l'angle au servomoteur.
  myServo.write(angle);
}
```

Lecture du fonctionnement

Traitement : mesure A0 → map() vers 0–180 → myServo.write(angle) → nouvelle position du servo.

L'objet myServo est lié à D9 par attach(). Dans la boucle, la conversion conserve les deux extrêmes : une lecture nulle donne 0°, tandis que 1023 donne 180°. Une lecture proche du milieu fournit un angle proche de 90°.

Prévision et relevé expérimental

Lecture A0	Angle prévu	Angle observé
0	0°	[À compléter]
Environ 512	Environ 90°	[À compléter]
1023	180°	[À compléter]

[Insérer les captures des positions 0°, 90° et 180°]

Analyse personnelle : comparer les positions observées aux trois consignes et commenter la continuité du mouvement. [...]

Bilan de la manipulation 4. La fonction `map ( )` adapte la plage du convertisseur analogique à la plage mécanique demandée. La bibliothèque Servo prend ensuite en charge le signal de commande sur D9. Les positions réelles doivent être consignées après simulation. Difficulté réelle : [à renseigner].

Problèmes rencontrés

Cette rubrique est réservée aux incidents réellement observés. Pour chaque problème, préciser le symptôme, la cause trouvée et la correction réalisée.

Problème n°1 : [...]

Cause : [...]

Correction : [...]

Problème n°2 : [...]

Cause : [...]

Correction : [...]

Synthèse finale

Ce TP réunit quatre usages fondamentaux de l'Arduino Uno. Le premier transforme une mesure analogique en décision binaire. Le deuxième organise plusieurs sorties dans le temps. Le troisième relie directement l'état électrique d'un bouton à une LED, avec une résistance de rappel. Le dernier convertit la position d'un potentiomètre en angle de servomoteur.

Les programmes respectent les affectations imposées : A0 et D8 pour le seuil, D13-D12-D11 pour le feu, D2 et D7 pour le bouton, puis A0 et D9 pour le servo. Les temporisations du feu sont exactement de 3000 ms, 1000 ms et 3000 ms. La partie restante consiste à réaliser les montages Proteus, recueillir les preuves visuelles et compléter l'analyse avec les résultats réellement mesurés.

Contrôle avant l'envoi

- ☐ compléter l'identité, la filière et l'année universitaire ;
- ☐ insérer des schémas Proteus nets et lisibles ;
- ☐ renseigner toutes les colonnes « observation » ;
- ☐ expliquer les écarts éventuels sans inventer de résultat ;
- ☐ remplacer les rubriques de difficultés par l'expérience réelle ;
- ☐ exporter sous le nom `Nom_Prenom_Examen_TP_Arduino.pdf`.